

《深度学习平台与应用》作业二

第一题：

- vanilla RNN的为什么会出现梯度消失或梯度爆炸的情况？

(言之有理都正确)

课件8第66页

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

总是小于1
梯度消失

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

梯度消失：

- 链式法则：通过乘以小于1的权重矩阵，导致梯度趋近于0。
- 激活函数：如 sigmoid 和 tanh 在饱和区域，导致梯度几乎为零。
- 长序列：随着时间步增加，模型难以学习长程依赖。

梯度爆炸

- 链式法则：通过乘以大于1的权重矩阵，导致梯度迅速增大。
- 长序列：在处理长序列时，梯度可能过快累积，导致不稳定。

- vanilla RNN 出现梯度爆炸该如何解决？

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

方法：梯度裁剪

- LSTM是如何缓解梯度消失的，它能彻底解决梯度消失吗？

解决方式：引入门控机制，输入门，遗忘门，输出门。动态调整信息流，避免了信息丢失，同时有连续的梯度流。

不能彻底解决：虽然LSTM大大减轻了梯度消失的问题，但并不能完全消除。特别是在处理非常长的序列时，仍然可能会出现梯度消失现象。

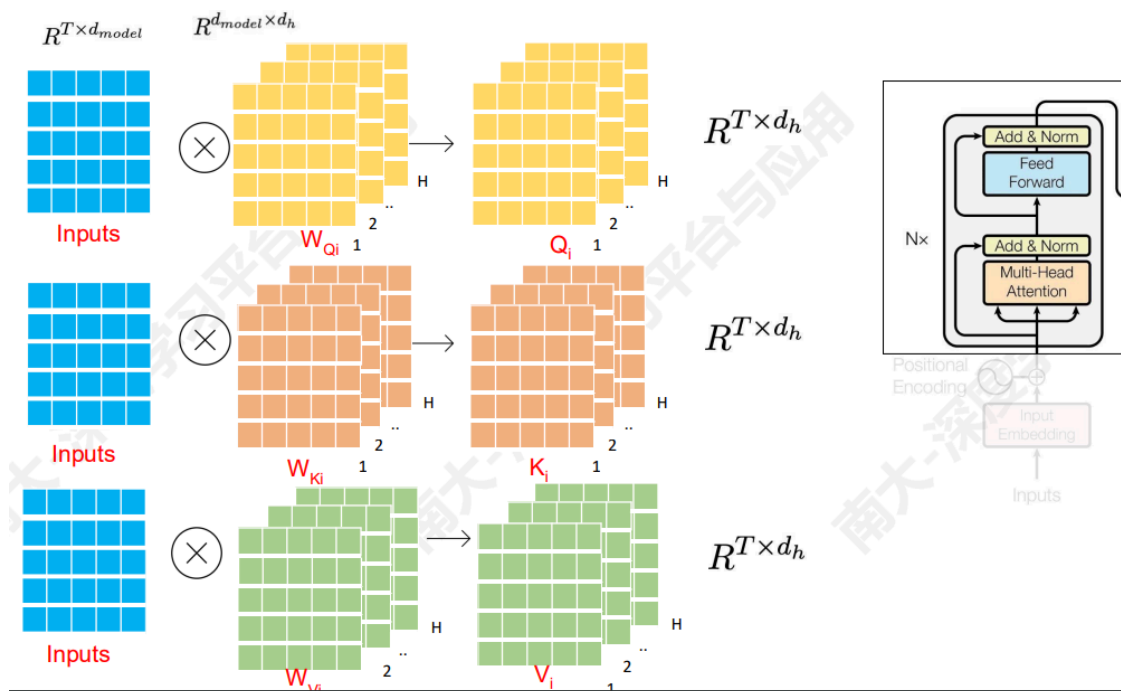
第二题:

- 问题1：完成下面self-attention的代码填空，并结合attention计算的数学公式解释你的做法。

```
1  import torch
2  import torch.nn.functional as F
3
4  # 假设输入是一个5个词，4维的词向量矩阵
5  x = torch.tensor([[1.0, 0.0, 1.0, 0.0],
6                    [1.0, 2.0, 1.0, 0.0],
7                    [0.0, 1.0, 0.0, 1.0],
8                    [0.0, 1.0, 0.0, 0.0],
9                    [1.0, 0.0, 1.0, 0.0]])
10
11 # 初始化查询、键和值的权重矩阵
12 w_Q = torch.randn(4, 4)
13 w_K = torch.randn(4, 4)
14 w_V = torch.randn(4, 4)
15
16 # 计算查询、键、值
17 Q = x.matmul(w_Q) # 查询
18 K = x.matmul(w_K) # 键
19 V = x.matmul(w_V) # 值
20
21 # 计算注意力得分
22 attention_scores = Q.matmul(K.T) / torch.sqrt(torch.tensor(K.size(1),
23 dtype=torch.float32)) # 缩放
24
25 # 应用softmax得到注意力权重
26 attention_weights = F.softmax(attention_scores, dim=-1)
27
28 # 加权和计算最终output
29 output = attention_weights.matmul(V)
30 print(output)
```

- 问题2：对比并简要解释多头注意力的实现方式。

知识点：课件9，第60页

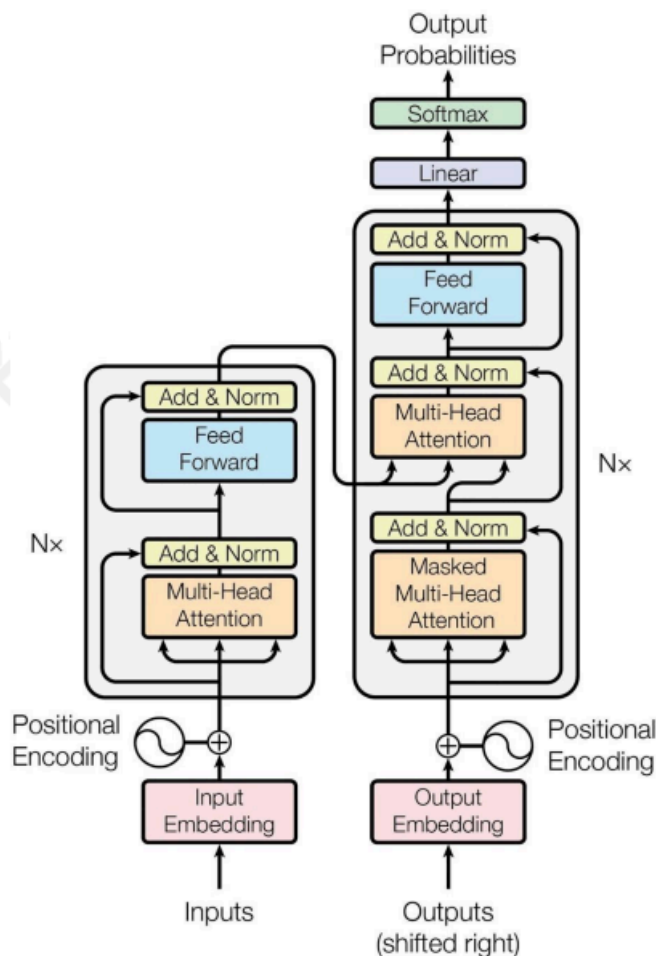


QKV的权重矩阵从单头注意力的一个增加为h个，每个的权重矩阵维度从单头的 $W_q \in \mathbb{R}^{d_{model} \times d_{model}}$ 变为 $W_q \in \mathbb{R}^{d_{model} \times d_{head}}$, 每个头单独计算注意力,互不影响, 满足 $d_{head} = \frac{d_{model}}{h}$, 然后将所有头的注意力输出拼接起来

$$Multi-head_Output = Concat(Output_1, Output_2, \dots, Output_h) \in \mathbb{R}^{T \times d_{model}}$$

拼接完得到的多头输出之后操作和单头注意力的做法一致。

- 问题3: Transformer结构中encoder和decoder的区别有哪些



1. 功能：Encoder接收输入序列，提取输入的特征并对其进行编码，Decoder生成输出序列，它依赖于编码器的输出以及已生成的输出部分。
2. 输入：Encoder通常能看到完整的序列，Decoder通常是自回归形式看到当前及以前的序列，不能看见未来的。
3. 注意力机制：Encoder使用自注意力，由自己产生，Decoder不仅有自注意力机制，而且还有交叉注意力部分，交叉注意力的Q、K来自于encoder
4. ...

第三题：

- 请简要描述R-CNN模型的工作流程。具体来说，R-CNN是如何进行目标检测的？简述R-CNN模型的主要步骤

知识点：课件10，第36页

1. 选择候选区域：

- 使用选择性搜索（Selective Search）算法生成一组候选区域（region proposals），这些区域可能包含目标物体。

2. 特征提取：

- 将每个候选区域输入到一个预训练的卷积神经网络（CNN）中，提取特征向量。每个候选区域生成一个固定长度的特征向量。

3. 分类:

- 将提取的特征向量送入支持向量机 (SVM) 进行分类, 得到每个候选区域的类别标签。

- 与传统的R-CNN相比, Fast R-CNN的最大改进是什么? 请简要说明Fast R-CNN如何避免了R-CNN中存在的冗余计算问题。

知识点: 课件10, 第38页

Fast R-CNN相较于传统R-CNN的最大改进在于特征计算的效率和冗余计算的避免。

改进:

1. 共享特征图:

- **Fast R-CNN**: 仅对整幅图像一次性计算特征图, 然后从这个特征图中提取各个候选区域的特征。

2. ROI Pooling层:

- Fast R-CNN新增了ROI Pooling层, 从完整的特征图中快速提取固定大小的特征用于目标分类和边界框回归, 减少了计算时间。

避免冗余计算

◦ 快速提取特征:

- 有了共享特征图, Fast R-CNN不再为每个候选区域重复计算CNN特征, 而是直接从已计算的特征图中获取信息, 显著降低了计算复杂度。